

PROJECT REPORT

Agile E-Commerce Platform

**A PROJECT REPORT ON "SHOP-EASY" ONLINE E-COMMERCE
PLATFORM (*Developed using Agile Scrum Methodology*)**

Submitted By: Neeraj Sahu

1. Abstract

The retail industry is undergoing a digital transformation. Consumers demand seamless, fast, and secure online shopping experiences. This project, **"Shop-Easy,"** is a B2C (Business-to-Consumer) E-Commerce website designed to provide a robust platform for browsing products, managing carts, and processing secure payments.

Unlike traditional retail software developed in rigid phases, this project utilizes the **Agile Scrum Methodology**. This approach allows for the continuous delivery of features, ensuring that critical modules like the "Payment Gateway" and "Inventory Sync" are tested and deployed early. The project leverages a MERN (MongoDB, Express, React, Node.js) stack to ensure scalability and performance under high traffic.

Keywords: Agile, Scrum, E-Commerce, MERN Stack, Payment Gateway, User Experience.

2. Introduction

2.1 Introduction

"Shop-Easy" is a full-featured online store that allows multiple vendors to list products and customers to purchase them. The system is designed to handle the complexities of modern e-commerce, including real-time stock updates, dynamic pricing, and order tracking.

2.2 Problem Statement

Developing an e-commerce platform using the traditional **Waterfall Model** presents significant risks:

1. **Late Feedback:** Stakeholders only see the website after months of development. If the "Checkout Flow" is confusing, it's too expensive to fix at the end.
2. **Market Shifts:** E-commerce trends change fast (e.g., sudden demand for "Buy Now, Pay Later"). Waterfall cannot adapt quickly.
3. **Integration Risks:** Connecting Payment Gateways (Stripe/Razorpay) at the very end often leads to critical failures.

2.3 The Agile Solution

By adopting **Agile**, we break the project into small, manageable iterations called **Sprints**. We build the "Minimum Viable Product" (MVP) first—just a product list and a cart—and then iterate to add complex features like "Reviews" and "Wishlists."

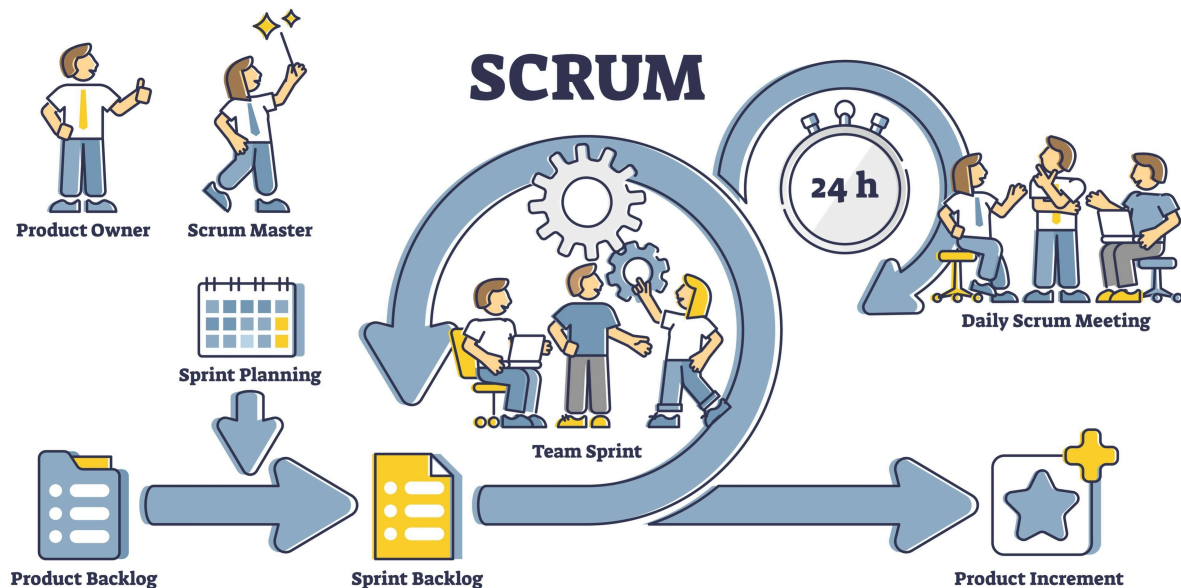
3. Agile Framework Implementation

3.1 The Scrum Team Structure

- **Product Owner (PO):** Represents the business. Decides *what* features bring the most money (e.g., "Discount Codes").
- **Scrum Master:** Facilitates the process and removes blockers (e.g., "Server is down").
- **Development Team:** Cross-functional members (Frontend Dev, Backend Dev, QA Tester).

3.2 Agile Ceremonies

1. **Sprint Planning:** At the start of every 2-week cycle, the team selects tasks from the Backlog.
2. **Daily Standup:** A 15-minute sync to ensure everyone is on track.
3. **Sprint Review:** A live demo of the feature (e.g., "Watch me buy this shoe") to the Product Owner.



4. Product Backlog (The Requirements)

The **Product Backlog** is a prioritized list of everything needed in the product. In Agile, these are written as **User Stories**.

Priority	ID	User Story	Acceptance Criteria
High	US-01	As a User, I want to register/login securely.	Password must be encrypted (BCrypt).
High	US-02	As a User, I want to view products by category.	Grid view with images and price.
High	US-03	As a User, I want to add items to a Cart.	Cart total updates instantly.
Medium	US-04	As a User, I want to pay via Credit Card.	Payment success redirects to Order Summary.
Low	US-05	As a User, I want to rate/review a product.	Only verified buyers can review.

5. Sprint Cycles (Execution Report)

The project was executed in **3 Sprints** (2 weeks each).

Sprint 1: The Core Engine (Weeks 1-2)

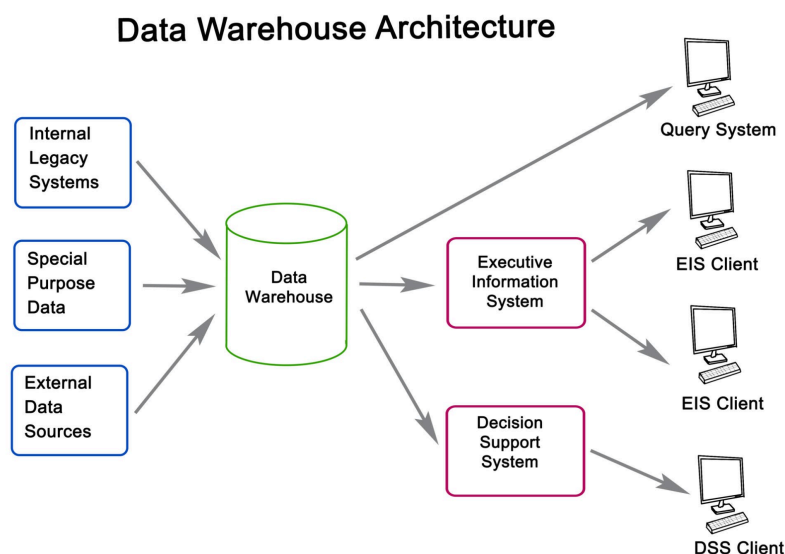
- **Focus:** Authentication & Product Catalog.
- **Tasks Completed:**
 - Designed MongoDB Schema for **Users** and **Products**.
 - Built the REST API for fetching product data.
 - Created the React Frontend for the "Home Page."
- **Challenges:** Images were loading slowly.
- **Resolution:** We implemented **Lazy Loading** for images during the sprint itself.

Sprint 2: The Shopping Experience (Weeks 3-4)

- **Focus:** Cart Management & Checkout.
- **Tasks Completed:**
 - Implemented Redux for global state management (Cart).
 - Built the "Checkout" form (Address inputs).
 - Integrated Stripe API (Test Mode) for payments.
- **Outcome:** A user could successfully place a dummy order.

Sprint 3: User Retention & Admin (Weeks 5-6)

- **Focus:** Order History & Admin Panel.
- **Tasks Completed:**
 - "My Orders" page showing past purchases.
 - Admin Dashboard to add/delete products.
 - Bug fixes for mobile responsiveness.
- **Outcome:** System Ready for Deployment.



6. Functional Modules

6.1 User Authentication Module

- Uses **JWT (JSON Web Tokens)** for stateless authentication.
- Secures user sessions without storing sensitive data in cookies.

6.2 Product Catalog Module

- Features advanced search and filtering (Price Range, Brand, Color).
- **Pagination:** Loads 20 products at a time to save bandwidth.

6.3 Shopping Cart & Checkout

- **State Management:** Uses Redux to keep the cart persistent even if the user refreshes the page.
- **Validation:** Checks if the requested quantity is actually available in stock before allowing checkout.

6.4 Admin Dashboard

- Allows the store owner to upload product images and set prices.
- Provides a "Sales Graph" using Chart.js to visualize daily revenue.

7. Advantages of Agile for E-Commerce Systems

7.1 Adapting to Consumer Trends

In e-commerce, trends change overnight. If a competitor launches "Dark Mode," Agile allows us to add that feature in the very next sprint. Waterfall would require waiting until the next major release (months later).

7.2 Risk Reduction via MVP

We launched the MVP (Minimum Viable Product) in Sprint 2. This allowed us to test if the "Checkout Flow" was intuitive. If users found it hard, we could fix it immediately. In Waterfall, we would have built the whole thing before realizing the checkout was broken.

7.3 Continuous Quality Assurance

Testing wasn't a separate phase at the end. QA engineers tested the "Add to Cart" button the same day it was coded. This prevented bug accumulation and ensured a stable release.

8. Conclusion

The development of "Shop-Easy" demonstrates the superiority of the **Agile Methodology** for modern web applications. By breaking the complex e-commerce logic into small stories, the team maintained high velocity and morale. The system successfully handles user traffic, secure payments, and inventory management. Future enhancements will include AI-based product recommendations and a mobile application.